Information Retrieval

WS 2022 / 2023

Lecture 2, Tuesday October 25th, 2022
(Ranking, Evaluation)

Prof. Dr. Hannah Bast
Chair of Algorithms and Data Structures
Department of Computer Science
University of Freiburg

UNI FREIBURG

Overview of this lecture

- Organizational
  - Your experiences with ES1 — Inverted index
  - Next Q&A in **two** weeks — Friday 04.11.2022 13:00
- Contents
  - Ranking — tf.idf and BM25, other tricks
  - Evaluation — Ground truth, Precision, AP, MAP, nDCG, BPref, Overfitting
  - **ES2:** Implement BM25, tune your search engine using a training benchmark, then evaluate on a test benchmark

    There will be a small competition for your enjoyment and motivation (in the form of a table on the Wiki)

Experiences with ES1 1/4

- Summary / excerpts
  - Interesting exercise, good start, quite doable for most, welcome opportunity to refresh rusty programming skills, a few with more problems

    "no major difficulties + lecture was easy to understand"

    "good start, not too hard but motivating, good to warm up"

    "hands-on experience in first week is exciting"

    "coding abilities more rusty than my bicycle chain"

    "I enjoyed it, but need to brush up my programming skills"

    "the two hours passed quickly + cool alarm sounds"

    "mention in HISinOne that Python is a prerequisite"

Experiences with ES1 2/4

- To the 10% or so for whom ES1 was very hard
  - We understand: it's not pleasant to be among those 10%
  - We are happy to help and support **everybody**

    Including those who have more problems than others
  - BUT you have to make an important decision:

    You have to **accep**t that this is (maybe much) more work for you than 6 ECTS

    Start **early** with the exercise sheets and reserve a sufficient amount of time for them

    We welcome your feedback, but please let us know **how you feel**, not your blame or what you think we are doing wrong

Experiences with ES1 3/4

- Results
  - Examples that work well (with brief explanation why):

shawshank redemption	highly specific words + popular movie
nolan	highly specific word ("Christopher Nolan")
Pokémon	highly specific word
lord of the rings	unspecific words, popular movie

  - Examples for queries that don't work well

2001 space odyssey	numbers not indexed
titanic	mentioned in irrelevant, but higher-ranked movies
harry potter	dito
pokemon	abstracts mention pokémon, not pokemon

Experiences with ES1 4/4

- Linear construction time?
  - I hope that nobody implemented something like this:

```
for line in file:
    …
    if record_id not in self.inverted_lists[word]:
        self.inverted_lists[word].append(record_id)
```

  - Then index construction on movies.tsv takes **very** long:

the "not in" takes linear time, not constant time

which means the whole loop takes **quadratic time**

Super-important piece of advice: **never** use built-in functions without understanding their time-complexity

Ranking 1/14

- Motivation
  - Queries often return many hits
  - Typically more than one wants to (or even can) look at

    For web search: often millions of documents

    But even for less hits a proper ranking is **key** to usability
  - So we want to have the most "relevant" hits first
  - **Problem:** how to measure how "relevant" something is?

Ranking 2/14

here: SUM

NOTE: a document containing only one of the words (like #34) can be ranked higher than documents containing two words

- Basic Idea
  - In the inverted lists, for each doc id also have a score

    university 17 0.5 , 53 0.2 , 97 0.3 , 127 0.8

    freiburg 23 0.1 , 34 0.8 , 53 0.1 , 127 0.7
  - While merging, **aggregate** the scores, then **sort** by score

    MERGED 17 0.5 , 23 0.1 , 34 0.8 , 53 0.3 , 97 0.3 , 127 1.5

    SORTED 127 1.5 , 34 0.8 , 17 0.5 , 53 0.3 , 97 0.3 , 23 0.1
  - The entries in the list are referred to as **postings**

    Above, it's only doc id and score, but a posting can also contain more information, e.g. the position of a word

Ranking 3/14

- Getting the top-k results
  - A full sort of the result list takes time $\Theta(n \cdot \log n)$, where n is the number of postings in the list
  - Typically only the top-k hits need to be displayed
  - Then a **partial sort** is sufficient: get the k largest elements, for a given k

    Can be computed in time $\Theta(n + k \cdot \log k)$

    k rounds of HeapSort yield time $\Theta(n + k \cdot \log n)$

    For constant k these are both $\mathbf{\Theta(n)}$

    For ES2, you can ignore this issue

Ranking 4/14

- Meaningful scores
  - How do we precompute good scores

    university 17 0.5 , 53 0.2 , 97 0.3 , 127 0.8

    freiburg 23 0.1 , 34 0.8 , 53 0.1 , 127 0.7
  - **Goal:** the score for the posting for doc D_i in the inverted list for word w should reflect the **relevance** of w in D_i

    In particular, the larger the score, the more relevant
  - **Problem:** relevance is somewhat subjective

    But it has to be done somehow anyway !

Ranking 5/14

- Term frequency (**tf**)
  - The number of times a word occurs in a document
  - **Problem:** some words are frequent in many documents, regardless of the content

university	… ,	**57**	**5**	, … … … ,	**123**	**2**	, …
of	… ,	**57**	**14**	, … … … ,	**123**	**23**	, …
freiburg	… ,	**57**	**3**	, … … … ,	**123**	**1**	, …
SCORE SUM	… ,	**57**	**22**	, … … … ,	**123**	**26**	, …

  - A word like "of" should not count much for relevance

Some of you observed that already while trying out queries for ES1

Ranking 6/14

$\log_b x = \frac{\ln x}{\ln b}$

- Document frequency (**df**)
  - The number of documents containing a particular word

    $\mathbf{df}_{\text{university}} = 16.384$ ($= 2^{14}$), $\mathbf{df}_{\text{of}} = 524.288$ ($= 2^{19}$), $\mathbf{df}_{\text{freiburg}} = 1.024$ ($= 2^{10}$)

    For simplicity, numbers are powers of 2, see below why

  - Inverse document frequency (**idf**)

    $\mathbf{idf} = \log_2 (N / df)$ N = total number of documents

    For the example df scores above and $N = 1.048.576 = 2^{20}$

    $\mathbf{idf}_{\text{university}} = 6$ ($\log_2(2^{20}/2^{14})$), $\mathbf{idf}_{\text{of}} = 1$ ($\log_2(2^{20}/2^{19})$), $\mathbf{idf}_{\text{freiburg}} = 10$ ($\log_2(2^{20}/2^{10})$)

    Note: The more frequent a word, the smaller the **idf**

    Without the $\mathbf{log_2}$, small differences in the value of **df** would have too much of an effect

Ranking 7/14

idf university = 6
idf of = 1
idf freiburg = 10

- Combining the two (**tf.idf**)
  - Reconsider our earlier **tf** only example

university	… ,	**57**	**5**	, … … … ,	**123**	**2**	, …
of	… ,	**57**	**14**	, … … … ,	**123**	**23**	, …
freiburg	… ,	**57**	**3**	, … … … ,	**123**	**1**	, …
SCORE SUM	… ,	**57**	**22**	, … … … ,	**123**	**26**	, …

  - Now combined with **idf** scores from previous slide

university	… ,	**57**	5·6 **30**	, … … … ,	**123**	2·6 **12**	, …
of	… ,	**57**	14·1 **14**	, … … … ,	**123**	23·1 **23**	, …
freiburg	… ,	**57**	3·10 **30**	, … … … ,	**123**	1·10 **10**	, …
SCORE SUM	… ,	**57**	**74**	, … … … ,	**123**	**45**	, …

Ranking 8/14

- Problems with tf.idf in practice
  - The idf part is fine, but the tf part has several problems
  - Let w be a word, and D_1 and D_2 be two documents
  - **Problem 1:** assume that D_1 is longer than D_2

    Then tf for w in D_1 tends to be larger than tf for w in D_2, because D_1 is longer, not because it's more "about" w
  - **Problem 2:** assume that D_1 and D_2 have the same length, and that the tf of w in D_1 is twice the tf of w in D_2

    Then it is reasonable to assume that D_1 is more "about" w than D_2, but just a little more, and not twice more

Ranking 9/14

- The **BM25** (best match) formula

  $tf^* = tf \frac{k+1}{k+tf}$

  – This tf.idf style formula has consistently outperformed other formulas in standard benchmarks over the years

  BM25 score = $tf^* \cdot \log_2 (N / df)$, where

  $b = 0 \Rightarrow \alpha = 1$

  tf* = $tf \cdot (k + 1) / (k \cdot \alpha + tf)$ $\alpha = (1 - b + b \cdot DL / AVDL)$

  tf = term frequency, DL = document length, AVDL = average document length (measured in number of words)

  – Standard setting for **BM25**: $k = 1.75$ and $b = 0.75$

  Binary: $k = 0$, $b = 0$; Normal tf.idf: $k = \infty$, $b = 0$

$tf^* = tf \cdot \frac{0+1}{0+tf} = 1$

$\lim_{k \to \infty} tf \cdot \frac{k+1}{k+tf}$

$= \lim_{k \to \infty} tf \cdot \frac{1 + \overset{\to 0}{1/k}}{1 + \underset{\to 0}{tf/k}} = tf$

Ranking 10/14

$$tf^* = tf \cdot \frac{k+1}{k \cdot \alpha + tf}$$

- Plausibility argument for BM25, part 1 (tf → tf*)
  - Start with the simple formula tf · idf
  - Replace tf by tf* such that the following properties hold:
    - tf* = 0 if and only if tf = 0 $\quad tf \cdot \underbrace{\frac{k+1}{k \cdot \alpha + tf}}_{\neq 0 \text{ for } k>0} = 0 \iff tf = 0$
    - tf* increases as tf increases [1]
    - tf* → fixed limit as tf → ∞ [2] $\quad \frac{1}{1+\frac{1}{x}}$
  - The "simplest" formula with these properties is
    - tf* = tf · (k + 1) / (k · α + tf) for any α (see next slide)

[1] $$tf^* = tf \cdot \frac{k+1}{k \cdot \alpha + tf} = \frac{k+1}{k \cdot \alpha / tf + 1}$$ increases as tf increases

[2] $$\lim_{tf \to \infty} tf \cdot \frac{k+1}{k \cdot \alpha + tf} = \lim_{tf \to \infty} \frac{k+1}{k \cdot \underbrace{\alpha / tf}_{\to 0} + 1} = k+1$$

Ranking 11/14

- Plausibility argument for BM25, part 2 (choice of α)
  - The α can be understood as a "normalization" of tf

    $tf^* = tf \cdot (k + 1) / (k \cdot \alpha + tf) = tf/\alpha \cdot (k + 1) / (k + tf/\alpha)$
  - BM25 normalizes by the length of the document
    - Full normalization: $\alpha = DL / AVDL$ … too extreme
    - Some normalization: $\alpha = (1 - b) + b \cdot DL / AVDL$

$b = 0: \alpha = 1$
$b = 1. \alpha = \frac{DL}{AVDL}$

In the literature, you find much more "theory" behind the BM25 formula. To me, it's not more convincing than the simple plausibility arguments from this slide and the last

Ranking 12/14

- Implementation advice
  - First compute the inverted lists with **tf** scores

    We already did that (implicitly) in Lecture 1 (not in ES1)
  - Along with that compute the document length (DL) for each document, and the average document length (AVDL)

    You can measure DL (and AVDL) via the number of words
  - Make a second pass over the inverted lists and replace the **tf** scores by **tf*** · **idf** scores

    $tf \cdot (k + 1) / (k \cdot (1 - b + b \cdot DL / AVDL) + tf) \cdot \log_2 (N / df)$

    Note that the **df** of a word is just the length (number of postings) in its inverted list

Ranking 13/14

- Further refinements
  - For ES2, play around with the BM25 parameters **k** and **b**
  - Boost results that match each query word at least once

    Warning: when you **restrict** your results to such matches, you miss relevant results, see your bad queries from ES1
  - Somehow take the popularity of a movie into account

    In the file on the Wiki, movies are sorted by popularity

    Note: some scores are already normalized (e.g. IMDb rating), others can't be taken directly as scores (e.g. IMDb #votes)

    A common heuristic to normalize scores with huge differences is to apply the **logarithm** (like we did for computing **idf**)
  - Anything else that comes to your mind and might help …

Ranking 14/14

- Other methods
  - There is a multitude of other sources / approaches for improving the quality of the ranking
  - **Example 1:** Using query logs and click-through data

    Who searches what and clicked on what … important signal in the ranking of big search engines like Google
  - **Example 2:** Learning to rank

    Using machine learning (more in a later lecture) to decide on the relative order of two documents

Evaluation 1/10

- Ground truth
  - For each query, we are given a ground truth in the form of the set of ids of all documents relevant for that query

    Query: matrix movies

    Relevant: 10, 582, 877, 10003
  - For ES2, we have built a ground truth for **22** queries

    Building a good and large enough ground truth is a common (and time-consuming) part in research in IR

  - On the next slides we will show how to use queries with a ground truth to measure the quality of a search engine

    A set of queries with a ground truth is therefore also referred to as a **benchmark**

Evaluation 2/10

- Precision (**P@k**)
  - The P@k for a given result list for a given query is the percentage of relevant documents among the top-k

    Query: matrix movies

    Relevant: 10, 582, 877, 10003

    REL NOT NOT REL REL NOT

    Result list: 582, 17, 5666, 10003, 10, 37, …

    P@1: 100% = 1

    P@2: 50% = 0.5

    P@3: 33% = 0.33…

    P@4: 50% = 0.5

    P@5: 60% = 0.6
  - **P@R** = P@k, where k = #relevant documents

Evaluation 3/10

- Average Precision (**AP**)
  - Let $R_1, \ldots, R_k$ be the sorted list of positions of the relevant documents in the result list of a given query
  - Then AP is the average of the k $P@R_i$ values

Query:	matrix movies
Relevant:	10, 582, 877, 10003
Result list:	REL 582, 17, 5666, REL 10003, REL 10, ..., REL 877
	1. 2. 3. 4. 5. 40.
$R_1, \ldots, R_4$:	1, 4, 5, 40
$P@R_1, \ldots, P@R_4$:	100%, 50%, 60%, 10%
AP:	(100% + 50% + 60% + 10%) / 4 = 55%

Note: for documents not in result list, just take $P@R_i = 0$

Evaluation 4/10

- Mean Precisions (**MP@k**, **MP@R**, **MAP**)
  - Given a benchmark with several queries + ground truth
  - Then one can capture the quality of a system by taking the **mean** (average) of a given measure over all queries

    MP@k = mean of the P@k values over all queries

    MP@R = mean of the P@R values over all queries

    MAP = mean of the AP values over all queries

    These are very common measures, which you will find in a lot of research papers on information retrieval

Evaluation 5/10

rel_i = relevance of i-th doc in result list

- Discounted Cumulative Gain (DCG, nDCG)
  - Sometimes relevance comes in more than one shade, e.g.

    0 = not relevant, 1 = somewhat rel, 2 = very relevant
  - There should be a <u>bonus</u> if very relevant documents are ranked higher than only somewhat relevant documents
  - **Cumulative gain:** $CG@k = \Sigma_{i=1..k}\, rel_i$
  - **Discounted CG:** $DCG@k = \Sigma_{i=1..k}\, rel_i / \log_2 (i + 1)$
  - Problem: CG and DCG are not normalized to [0,1]
  - Solution: normalize by maximally achievable value
  - **Ideal DCG:** iDCG@k = DCG@k of ideal ranking
  - **Normalized DCG:** nDCG@k = DCG@k / iDCG@k

Evaluation 6/10

$$DCG@5 = \sum_{i=1}^{5} \frac{rel_i}{\log_2(i+1)}$$

- Discounted Cumulative Gain (DCG, nDCG), example
  - Consider the following result list and relevances, assuming only 3 relevant documents overall (for this query)

i				IDEAL
1	Hit #1:	relevant	1	2
2	Hit #2:	not relevant	0	1
3	Hit #3:	very relevant	2	1
4	Hit #4:	relevant	1	0
5	Hit #5:	not relevant	0	0

  - Then **DCG@5** = $1 + 0 + \frac{2}{\log_2 4} + \underbrace{\frac{1}{\log_2 5}}_{=0.43} + 0 = 2.43$
  - And **iDCG@5** = $2 + \underbrace{\frac{1}{\log_2 3}}_{=0.63} + \underbrace{\frac{1}{\log_2 4}}_{=2} + 0 + 0 = 3.13$
  - Hence **nDGC@5** = $2.43 / 3.13 = 77,6\,\%$

Evaluation 7/10

A common approach in competitions is **pooling**: jugde only the top-k results from each participant

- Binary preference (bpref)
  - Sometimes we have relevance judgements only for a subset of all the documents

    Typically for very large datasets, where it is infeasible to check all the documents for relevance
  - Then measure whether judged relevant documents come before judged non-relevant documents
  - bpref = $1/|R| \cdot \Sigma_{r \in RR} (1 - |NR(r)| / \min(|R|, |N|))$ where

    R = docs judged relevant
    N = docs judged non-relevant
    RR = docs judged relevant in result list
    NR(r) = docs from the |R| top-ranked non-relevant before r

Evaluation 8/10

- Binary preference (bpref), example

  $\min(|R|, |N|) = 3$

  – Consider the following result list

  #1: ???
  #2: not relevant
  #3: relevant
  #4: not relevant
  #5: ???
  #6: ???
  #7: not relevant
  #8: relevant
  #9: ???

  Not in list:

  One more relevant document
  Ten more non-relevant documents

RR = 2

|R| = 3
|N| = 13

$1 - \frac{NR(\#3)}{\min(\dots)}$ ~~NR(#3)~~ $= 1 - \frac{1}{3} = \frac{2}{3} = 66\%$

$1 - \frac{NR(\#8)}{\min(\dots)}$ ~~NR(#8)~~ $= 1 - \frac{3}{3} = 0 = 0\%$

bpref $= \frac{2/3 + 0 + 0}{3}$

$= 2/9 = 22\%$

Evaluation 9/10

- Overfitting
  - Tuning parameters / methods to achieve good results on a given benchmark is called **overfitting**

    In an extreme case: for each query from the benchmark, output the list of relevant docs from the ground truth
  - In a realistic environment (real search engine or competition), one is given a **training** set for development

    The actual evaluation is on a **test** set, which must not be used or is not available during development

    For ES2, we explicitly provide a training benchmark for development, and a test benchmark to measure the quality of your system once you are done with tuning

Evaluation 10/10

- Other measures
  - On the previous slides, the ground truth for each query was a **set** and the results we evaluated were **ranked lists**
  - Why don't we let our search engine also output a set and measure how similar it is to the ground truth?

    Because there is no natural cut-off value
  - Why isn't our ground truth also a ranked list and we measure the similarity of the two rankings?

    Because the set of relevant documents usually doesn't have a full order (though there might be levels of relevance)
  - Why don't we evaluate the accuracy of the scores instead of just the order that is implied by the scores?

    The absolute values of the scores don't mean much, it's mainly about the order implied by the scores

References

- In the Manning/Raghavan/Schütze textbook

  Section 6: Scoring and Term Weighting

  Section 8: Evaluation in Information Retrieval

- Relevant Papers

  Probabilistic Relevance: BM25 and Beyond **FnTIR 2009**

  Test Collection Based Evaluation of IR Systems **FnTIR 2010**

- Relevant Wikipedia articles

  http://en.wikipedia.org/wiki/Okapi_BM25

  https://en.wikipedia.org/wiki/Information_retrieval#Performance_and_correctness_measures